

Resource-Constrained CNN for Edge Image Classification

EN3150 Assignment 03

Xception

WIJETHUNGA W. M. D. P	–	220727G
WIJERATNE E.N.K	–	220717C
GALLAGE A.H.	–	220172A
DISANAYAKA J.R.S.	–	220132D

EN3150 – Pattern Recognition

Code repository

https://github.com/dpiyumal2319/pattern-recognition-cnn-edge

Date

October 19, 2026

Contents

1 Task 1: Data Preparation **2**
- 1.1 Dataset . . . 2
- 1.2 Preprocessing and augmentation . . . 2

2 Task 2: Custom Architecture Design **5**
- 2.1 Custom architecture design . . . 5
  - 2.1.1 Model A: standard CNN . . . 5
  - 2.1.2 Model B: depthwise separable CNN . . . 6
  - 2.1.3 Compute saving and profiler cross-check . . . 6
- 2.2 Hardware-aware activation-function choice . . . 7
  - 2.2.1 ReLU6 and low-precision deployment . . . 7
  - 2.2.2 Output probabilities . . . 8

3 Task 3: Optimizer Selection and Tuning **9**
- 3.1 Optimizer comparison protocol . . . 9
- 3.2 Effect of the momentum coefficient . . . 10

4 Task 4: Custom Model Training and Evaluation **11**
- 4.1 Custom-model training and evaluation . . . 11
  - 4.1.1 Convergence control . . . 14

5 Task 5: Lightweight State-of-the-Art Models **15**
- 5.1 Transfer-learning protocol . . . 15
- 5.2 Results . . . 15

6 Task 6: Final Comparison and Trade-offs **21**
- 6.1 Model B against lightweight SOTA models . . . 21
- 6.2 Trade-offs: accuracy, memory and compute . . . 21
- 6.3 Conclusion . . . 23

A Generated Layer Tables **23**

B Training Hyperparameters **24**

References **25**

1 Task 1: Data Preparation

1.1 Dataset

We use EuroSAT [1], a land-use and land-cover dataset containing 27,000 labelled, georeferenced Sentinel-2 image patches across ten classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, and SeaLake. Although the original dataset includes all 13 Sentinel-2 spectral bands, this work uses the RGB version, which contains the three visible bands stored as 8-bit images. The dataset is available under the MIT licence and was downloaded through `torchvision` [2] version 0.20 or later, which uses the maintained Hugging Face mirror. Because each class represents a land-cover category viewed from an overhead perspective, orientation changes such as flips and 90-degree rotations do not alter the class label, which supports the augmentation choices described in Section 1.2.

Input resolution. All EuroSAT images have a native resolution of 64×64 pixels. Since the assignment specifies a maximum input resolution of 64×64 for the low-resource setting, the dataset already satisfies this requirement and no downscaling is needed. The custom models therefore use the original images directly, without reducing their spatial resolution. Figure 1 shows one native sample from each class. The only resizing beyond the native resolution is an upsampling step to 224×224 for the supplementary SOTA resolution experiment in Task 5, where the pretrained models are also evaluated at their standard design resolution.

Split. The dataset was divided once into training, validation, and test sets using the required 70/15/15 proportions. The split was stratified by class, so each class was divided independently and the class distribution remains consistent across all three sets. Class sizes range from 2,000 to 3,000 images (Table 1), and these sizes allow the 70/15/15 proportions to be applied exactly. Within each class, image files were sorted by name and then shuffled using a fixed seed of 42. This makes the split reproducible and ensures that the same partition is obtained across different machines.

One partition for every experiment. The train, validation, and test assignment for each image is recorded in the committed manifest, `data/splits.csv`. All experiments use this same file, including the custom models, optimizer comparison, and fine-tuned SOTA models, rather than generating new splits. This keeps the data partition fixed across experiments and satisfies the requirement that the SOTA models use the same splits as the custom networks. Table 1 is generated directly from this manifest. The validation set is used for model and optimizer selection, while the test set is reserved for final evaluation after training.

1.2 Preprocessing and augmentation

Standardisation. Each RGB channel is standardised as

$$x'_c = \frac{x_c - \mu_c}{\sigma_c}, \tag{1}$$

where pixel values are first scaled to $[0, 1]$, and μ_c and σ_c denote the mean and population standard deviation of channel c. The statistics are computed from the EuroSAT training split and the same values are then applied to the training, validation, and test sets. This places the three colour channels on a comparable scale before they enter the network. The same EuroSAT-derived statistics are also used for the pretrained models in Task 5. Although those models were originally trained on ImageNet using ImageNet channel statistics [3], using a common EuroSAT preprocessing pipeline keeps the input treatment consistent across the models being compared.

Table 1: Images per class in each set, generated from the committed split manifest `data/splits.csv`.

Class	Train	Validation	Test	Total
AnnualCrop	2,100	450	450	3,000
Forest	2,100	450	450	3,000
HerbaceousVegetation	2,100	450	450	3,000
Highway	1,750	375	375	2,500
Industrial	1,750	375	375	2,500
Pasture	1,400	300	300	2,000
PermanentCrop	1,750	375	375	2,500
Residential	2,100	450	450	3,000
River	1,750	375	375	2,500
SeaLake	2,100	450	450	3,000
Total	18,900	4,050	4,050	27,000
Share	70.0%	15.0%	15.0%	100.0%

Statistics from the training set only. The normalisation statistics are estimated only from the training partition. Computing μ_c and σ_c over all 27,000 images would allow information from the validation and test sets to influence preprocessing, which would weaken the independence of the final evaluation. The split is therefore fixed first, after which the channel means and standard deviations are calculated from the training images only, using the native-resolution images before augmentation. These same values are then applied unchanged to the validation and test sets. The statistics are cached after computation so that every experiment uses the same normalisation values.

Augmentation. Data augmentation is applied only to the training set. Each training image is independently subjected to a horizontal flip with probability 0.5, a vertical flip with probability 0.5, and a rotation chosen uniformly from 0°, 90°, 180°, and 270°. These transformations are sampled again each epoch, producing the eight possible orientations of a square image over time (Figure 1).

These operations preserve the EuroSAT class labels because the images are captured from an overhead viewpoint, where image orientation has no fixed physical meaning. A highway, river or crop field remains the same land-cover class after rotation or reflection. Restricting rotations to multiples of 90° also avoids interpolation and padding, since the transformation only rearranges pixels on the square image grid.

Other common augmentations were not used. Random cropping and rescaling can alter the apparent scale of structures and may remove useful content from the small 64×64 images. Colour jitter was also avoided because colour and spectral appearance help distinguish classes such as vegetation, water, and bare or built-up surfaces.

Evaluation preprocessing and class balance. Validation and test images are converted to tensors and standardised using the statistics computed from the training set, with no random augmentation applied. This keeps the evaluation inputs fixed and makes evaluation deterministic. With class sizes ranging from 2,000 to 3,000 images (Table 1), EuroSAT has a mild class imbalance. There was no under sampling or synthetic minority generation used since the largest class is only 1.5 times the size of the smallest. Stratified splitting preserves the same class proportions across the training, validation, and test sets, while macro averaged precision and recall are reported together with accuracy so that performance on smaller classes contributes equally to the evaluation.



Figure 1: One training image from each EuroSAT class before and after the configured online augmentation. EuroSAT images are natively 64×64 pixels, so these are shown at their original resolution.

2 Task 2: Custom Architecture Design

2.1 Custom architecture design

Both custom networks take a 64×64 RGB image and output ten class logits. Each feature-extraction stage has a valid 3×3 convolutional operation with stride 1 and no padding, followed by ReLU and 2×2 max pooling with stride 2. Because of this, the spatial size goes through $64 \to 62 \to 31 \to 29 \to 14 \to 12 \to 6$ over the three stages. Both networks use channel widths of 32, 64 and 128 and hidden fully connected widths of 128 and 64. The only differences are that Model B replaces Model A's standard convolutions with depthwise separable convolutions, and replaces its flattened head with global average pooling (GAP).

All convolutional and dense layers have biases, and there is no batch normalisation in either model. During training, dropout with probability 0.2 is applied right before each hidden dense layer, and it is turned off for validation and inference. The final dense layer outputs raw logits, so there is no ReLU or softmax after it.

A standard convolution has

$$P_{\text{conv}} = (K_h K_w C_{\text{in}} + 1) C_{\text{out}}, \tag{2}$$

where the added one is the bias of each output channel. A depthwise and pointwise pair instead has

$$P_{\text{sep}} = (K_h K_w + 1) C_{\text{in}} + (C_{\text{in}} + 1) C_{\text{out}}. \tag{3}$$

A dense layer has $(N_{\text{in}} + 1) N_{\text{out}}$ parameters. For one image, we count convolution MACs as $K_h K_w C_{\text{in}} C_{\text{out}} H_{\text{out}} W_{\text{out}}$ and dense MACs as $N_{\text{in}} N_{\text{out}}$, and bias additions are not counted. Max pooling, ReLU, dropout, flatten and GAP have no trainable parameters. Pooling and averaging operations are also left out of the MAC total, because the project profiler does not count them and the hand derivation should match the profiler exactly.

2.1.1 Model A: standard CNN

Model A flattens the final feature map into $128 \times 6 \times 6 = 4{,}608$ values before the first dense layer, and Table 2 derives the parameters and MACs of every layer from these shapes.

Table 2: Hand-derived parameters and MACs for Model A.

Layer	Output	Parameter arithmetic	MAC arithmetic
Conv 1	$32 \times 62 \times 62$	$(3 \times 3 \times 3 + 1) \times 32 = 896$	$3 \times 3 \times 3 \times 32 \times 62^2 = 3{,}321{,}216$
MaxPool 1	$32 \times 31 \times 31$	0	0 (not profiled)
Conv 2	$64 \times 29 \times 29$	$(3 \times 3 \times 32 + 1) \times 64 = 18{,}496$	$3 \times 3 \times 32 \times 64 \times 29^2 = 15{,}501{,}312$
MaxPool 2	$64 \times 14 \times 14$	0	0 (not profiled)
Conv 3	$128 \times 12 \times 12$	$(3 \times 3 \times 64 + 1) \times 128 = 73{,}856$	$3 \times 3 \times 64 \times 128 \times 12^2 = 10{,}616{,}832$
MaxPool 3	$128 \times 6 \times 6$	0	0 (not profiled)
Flatten	4,608	0	0
Dense 1	128	$(4{,}608 + 1) \times 128 = 589{,}952$	$4{,}608 \times 128 = 589{,}824$
Dense 2	64	$(128 + 1) \times 64 = 8{,}256$	$128 \times 64 = 8{,}192$
Output	10	$(64 + 1) \times 10 = 650$	$64 \times 10 = 640$
Total		692,106	30,038,016

The convolutional stack has only 93,248 parameters, while the dense head has 598,858, which is 86.5% of the total. So the head is the part that Model B has to cut if it is to meet its parameter budget.

2.1.2 Model B: depthwise separable CNN

In each Model B stage, the depthwise step applies one 3×3 filter to each input channel, and the pointwise 1×1 step then mixes those channels. There is only one ReLU per stage, and it comes after the pointwise step. Table 3 derives the depthwise and pointwise costs of each stage separately.

Table 3: Hand-derived parameters and MACs for Model B. DW and PW denote depthwise 3×3 and pointwise 1×1 convolution.

Layer	Output	Parameter arithmetic	MAC arithmetic
DW 1	$3 \times 62 \times 62$	$(3 \times 3 + 1) \times 3 = 30$	$3 \times 3 \times 3 \times 62^2 = 103{,}788$
PW 1	$32 \times 62 \times 62$	$(3 + 1) \times 32 = 128$	$3 \times 32 \times 62^2 = 369{,}024$
MaxPool 1	$32 \times 31 \times 31$	0	0 (not profiled)
DW 2	$32 \times 29 \times 29$	$(3 \times 3 + 1) \times 32 = 320$	$3 \times 3 \times 32 \times 29^2 = 242{,}208$
PW 2	$64 \times 29 \times 29$	$(32 + 1) \times 64 = 2{,}112$	$32 \times 64 \times 29^2 = 1{,}722{,}368$
MaxPool 2	$64 \times 14 \times 14$	0	0 (not profiled)
DW 3	$64 \times 12 \times 12$	$(3 \times 3 + 1) \times 64 = 640$	$3 \times 3 \times 64 \times 12^2 = 82{,}944$
PW 3	$128 \times 12 \times 12$	$(64 + 1) \times 128 = 8{,}320$	$64 \times 128 \times 12^2 = 1{,}179{,}648$
MaxPool 3	$128 \times 6 \times 6$	0	0 (not profiled)
GAP and flatten	128	0	0 (not profiled)
Dense 1	128	$(128 + 1) \times 128 = 16{,}512$	$128 \times 128 = 16{,}384$
Dense 2	64	$(128 + 1) \times 64 = 8{,}256$	$128 \times 64 = 8{,}192$
Output	10	$(64 + 1) \times 10 = 650$	$64 \times 10 = 640$
Total		36,968	3,725,196

GAP reduces the $128 \times 6 \times 6$ tensor to 128 channel means before the dense stack, so Model B does not need the $4{,}608 \to 128$ layer that dominates Model A.

The separable blocks have 11,550 parameters and the GAP-based dense head has 25,418, which puts Model B 63,032 parameters below the 100,000-parameter limit. Keeping a flattened head would leave a 589,952-parameter first dense layer, and that layer alone is already over the budget. Narrowing the channels might also have brought the count under the limit, but it would reduce the capacity of the feature extractor, so we used GAP and kept the same channel widths as Model A.

2.1.3 Compute saving and profiler cross-check

Table 4 compares only the convolution substitution, at the same input and output shapes. Over the three stages, depthwise separable convolution reduces the convolution MACs from 29,439,360 to 3,699,980, which is a 7.96× reduction. The reduction gets larger as the channel count grows. For $256 \to 256$ channels, the exact cost per output location is $9 \times 256 \times 256 = 589{,}824$ MACs for a standard convolution and $9 \times 256 + 256 \times 256 = 67{,}840$ for the separable version, which gives an 8.69× reduction.

Table 4: Direct MAC comparison between standard and depthwise separable convolution at each common stage shape.

Stage ($C_{\text{in}} \to C_{\text{out}}$)	Standard	Separable	Reduction
$3 \to 32$, 62×62	3,321,216	472,812	7.02×
$32 \to 64$, 29×29	15,501,312	1,964,576	7.89×
$64 \to 128$, 12×12	10,616,832	1,262,592	8.41×
Convolution total	29,439,360	3,699,980	7.96×

When the classifier heads are included, Model B has 18.72× fewer trainable parameters and 8.06× fewer MACs per image than Model A, which are reductions of 94.66% and 87.60%. We did not measure the resulting speedup, and actual inference time also depends on memory traffic, kernel support and the operations left out of the MAC count.

The forward-hook profiler gives 692,106 parameters and 30,038,016 MACs for Model A, and 36,968 parameters and 3,725,196 MACs for Model B, which agree exactly with the hand derivations.

2.2 Hardware-aware activation-function choice

Models A and B use ReLU, $f(x) = \max(0, x)$, after each convolutional stage and each hidden fully connected layer. It passes positive values through unchanged and sets negative values to zero, so it does not saturate for positive inputs. Negative inputs have zero gradient, so a unit that stays inactive can stop learning, which is the dying-ReLU problem. ReLU outputs are also not zero-centred.

Each model applies ReLU to

$$32(62^2) + 64(29^2) + 128(12^2) + 128 + 64 = 195{,}456 \tag{4}$$

hidden values per image. Arm's CMSIS-NN Q7 reference implementation compares each value with zero and sets the negative ones to zero. On DSP-capable M-profile cores, its vector path masks four signed 8-bit values in each loop iteration [4, `arm_relu_q7.c`, ll. 50–101]. The operation needs no coefficients or lookup table. Loads, stores and loop control still add to the cycle count, and that count depends on the core and the compiler.

Sigmoid and tanh, defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \qquad \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}, \tag{5}$$

both need exponentials or an approximation of them. They also saturate at extreme inputs, where their gradients approach zero. In CMSIS-NN, the 16-bit versions of these functions use a 256-entry `uint16_t` sigmoid table. Inputs inside the table range need two table reads and an integer interpolation, and inputs outside it take a saturation branch. The table takes $256 \times 2 = 512$ bytes of read-only memory, before counting code and temporary values [4, `arm_nn_activation_s16.c`, ll. 51–117; `arm_nn_tables.h`, l. 40]. Using these functions at every hidden site would mean 195,456 such evaluations per image. These integer kernels are only examples of embedded implementations, and our own experiments use the floating-point kernels in PyTorch.

The cost of floating-point work also depends on the core. Arm describes the FPU on Cortex-M4 and Cortex-M7 as optional, and compares hardware floating point with emulation using integer operations [5, pp. 4–5]. Our activation comparison is therefore analytical, and we did not measure inference latency on a microcontroller.

GELU's exact form, $x\Phi(x)$, uses the Gaussian cumulative distribution, which can be expressed using the error function, and its common approximation uses tanh [6]. Leaky ReLU and PReLU keep a nonzero gradient on the negative branch when the slope is nonzero, but they need a multiply on that branch, and PReLU also has to learn and store the slope α. We chose ReLU because its inference operation is the simplest of these, but we did not compare the activation functions experimentally.

2.2.1 ReLU6 and low-precision deployment

MobileNetV2 uses $\mathrm{ReLU6}(x) = \min(\max(0, x), 6)$ because a bounded activation holds up better in low-precision computation [7, Sec. 4]. The upper bound limits the range that has to be

represented, so the integer levels can be spaced more closely inside it. CMSIS-NN also provides a signed 8-bit ReLU6 operator, but our custom models run in floating point and quantisation is outside the scope of this project. We therefore kept ordinary ReLU, so values above six still reach the next layer.

2.2.2 Output probabilities

Softmax converts the ten output logits into normalised probabilities,

$$p(y = C_k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^{10} \exp(z_j)}. \tag{6}$$

This needs only ten exponentials per image. Applying an exponential-based activation to all 195,456 hidden values instead would involve about 19,546 times as many values as the output softmax.

During training, the models return raw logits to `CrossEntropyLoss`. At inference, the predicted class is the index of the largest logit, and softmax does not change this ordering. Softmax is needed only when normalised probabilities have to be reported, and on its own it does not make those probabilities calibrated.

3 Task 3: Optimizer Selection and Tuning

3.1 Optimizer comparison protocol

We trained Model B with three optimizer configurations, Adam at $\alpha = 10^{-3}$, plain stochastic gradient descent (SGD) at $\alpha = 10^{-2}$, and SGD at the same learning rate with momentum $m = 0.9$. The model, split manifest, data order and augmentation were the same in every run. Each run used seed 42, batch size 128, 30 epochs, zero weight decay and a fixed learning rate with no scheduler, and the sweep script checks that these shared settings match before training starts. The saved Adam configuration has `device: auto` while the SGD configurations have `device: cuda`, but all three profiles record the same Tesla T4.

SGD updates the weights by

$$w \leftarrow w - \alpha \frac{\partial L(w)}{\partial w}. \tag{7}$$

With momentum, the update becomes

$$v \leftarrow mv - \alpha \frac{\partial L(w)}{\partial w}, \qquad w \leftarrow w + v. \tag{8}$$

Comparing $m = 0$ and $m = 0.9$ at the same SGD learning rate therefore isolates the effect of momentum. Adam instead scales the step of each parameter using running first and second moments of the gradients, and we used its default coefficients $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$.

We did not tune either learning rate for this problem. Adam uses $\alpha = 10^{-3}$, the value recommended by its authors, who report that its hyperparameters typically need little tuning [8]. SGD uses $\alpha = 10^{-2}$ as a starting value. The different values account for Adam's per-parameter adaptation and SGD's global step size; a shared numerical learning rate would not give equivalent updates.

Schmidt et al. found in their benchmark that trying several optimizers with default settings performed about as well as tuning one fixed optimizer [9]. This supports the baseline comparison used here. Plain SGD may improve with a separately tuned learning rate. Each configuration was trained with only one seed, so these results hold only for the recorded configurations and training budget.

We selected Adam at $\alpha = 10^{-3}$ for the custom-model runs because it had the lowest validation loss in Table 5. It also had the highest validation accuracy at the selected checkpoint and the highest test accuracy of the three runs. SGD with momentum reduced validation loss faster than plain SGD over the 30 epochs. For every run, the reported metrics come from the checkpoint with the lowest validation loss, and test accuracy was used only for the final evaluation.

Optimizer	LR	SGD m	Best val. loss	Val. acc. (%)	Test acc. (%)	Epoch (s)
Adam	0.001	–	0.427	84.42	83.75	11.61
SGD	0.01	0.0	1.996	23.14	23.48	11.12
SGD + momentum	0.01	0.9	0.471	83.56	82.86	11.02

Table 5: Optimizer comparison for Model B. All runs use the same Tesla T4, split, seed, data order, batch size, augmentation, and 30-epoch budget. Validation accuracy is from the lowest-validation-loss checkpoint, not necessarily the maximum-accuracy epoch. m applies only to SGD, and Adam uses its own moment coefficients. Epoch time is the mean training-pass time, excluding validation.

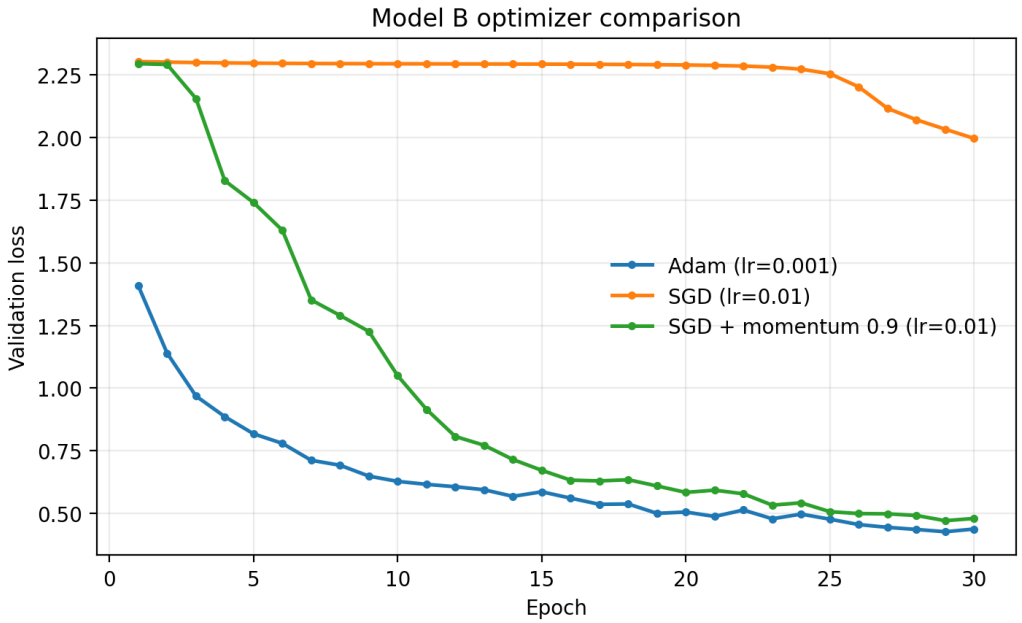


Figure 2: Validation-loss comparison for Adam, plain SGD, and SGD with momentum on Model B under an otherwise identical training budget.

3.2 Effect of the momentum coefficient

We then trained Model B using SGD with a fixed learning rate of $\alpha = 10^{-2}$ and $m \in \{0, 0.5, 0.9, 0.99\}$. The split, seed, data order, augmentation, batch size, initialisation and 30-epoch budget were identical in all four runs, and all four used an NVIDIA GeForce RTX 3060 Laptop GPU, so their epoch times cannot be compared directly with the T4 runs in Table 5. Figure 3 plots the four runs, which are recorded as `momentum_m000`, `momentum_m050`, `momentum_m090` and `momentum_m099`.

In the momentum update,

$$v \leftarrow mv - \alpha \frac{\partial L(w)}{\partial w}, \qquad w \leftarrow w + v, \tag{9}$$

m sets how much of the previous velocity is kept. When successive gradients have a consistent sign, they accumulate in v and the weights move further in that direction. Across a narrow loss ravine, gradient components that alternate in sign cancel part of the stored velocity, so the side-to-side motion is damped.

With $m = 0$, the update has no memory and validation loss stays high for the whole epoch budget. Setting $m = 0.5$ makes the loss start falling sooner, but it is still high after 30 epochs. The $m = 0.9$ run falls with small epoch-to-epoch fluctuations and reaches the lowest validation loss of the four. At $m = 0.99$ the loss falls fastest at first, but it fluctuates in the later epochs and finishes above its selected minimum. Keeping more of the past velocity could explain these fluctuations, although the loss curves alone cannot confirm it.

We selected $m = 0.9$ because it had the lowest validation loss. The $m = 0.99$ run had a slightly higher test accuracy, but that did not affect the choice because test accuracy is not used for selection. This ranking comes from one run per setting, and we have not measured whether it holds across other random seeds.

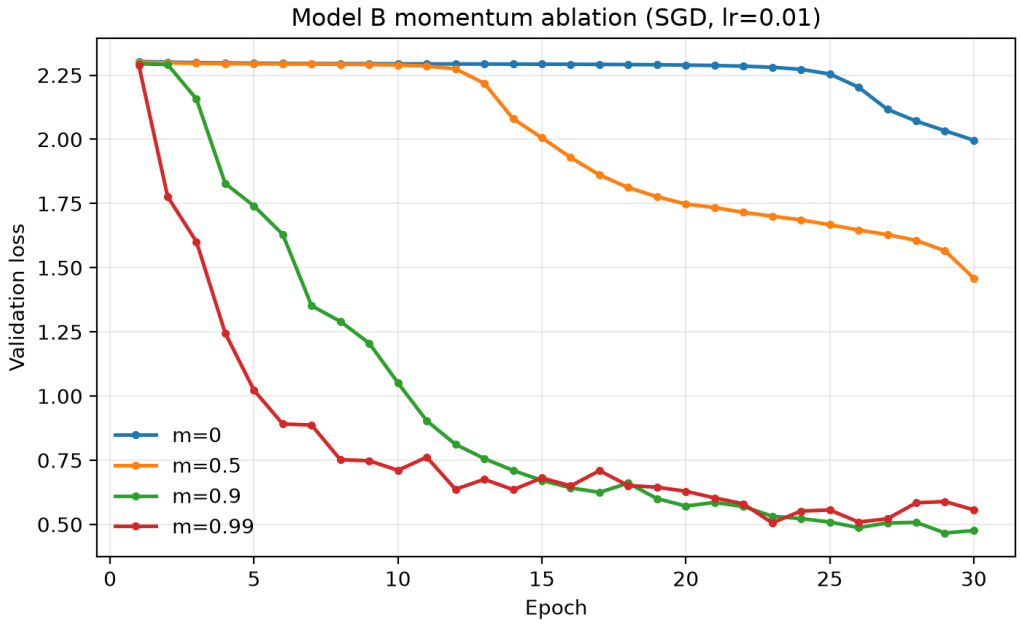


Figure 3: Validation-loss curves for Model B with SGD at $\alpha = 10^{-2}$. Momentum is the only methodological variable across the four 30-epoch runs.

4 Task 4: Custom Model Training and Evaluation

4.1 Custom-model training and evaluation

Both custom networks were trained for 30 epochs with Adam on the same stratified split, random seed, augmentation pipeline, batch size, and Tesla T4 device. Checkpoint selection used the lowest validation loss, and the test split was evaluated only after that selection. Figure 4 therefore compares like with like rather than mixing devices or training budgets.

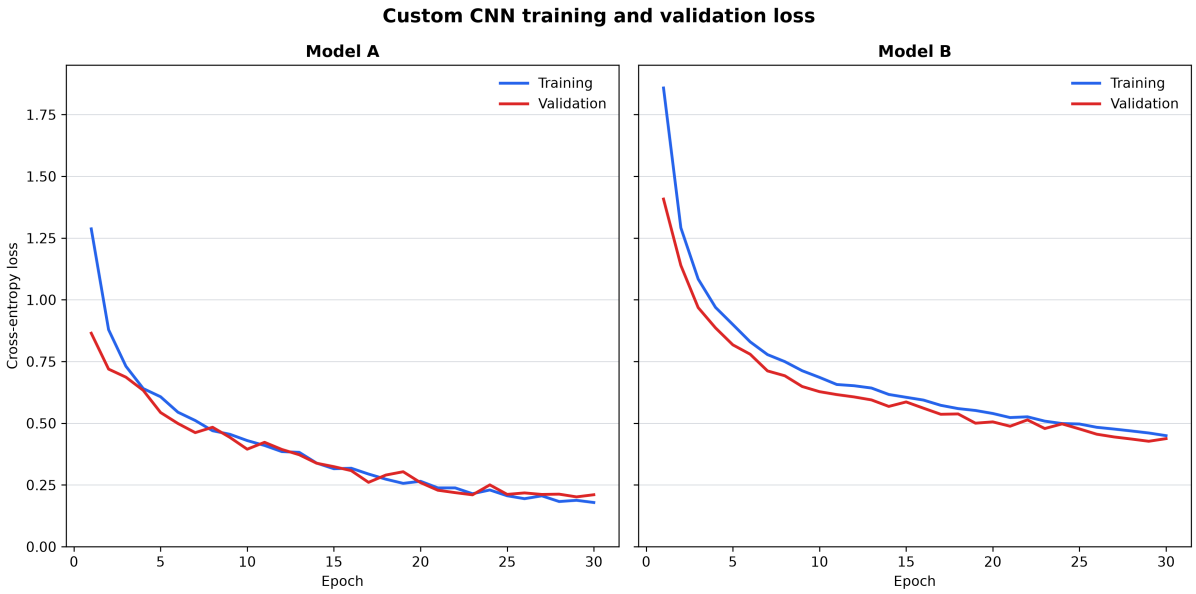


Figure 4: Training and validation cross-entropy loss for Model A and Model B. The panels share a common vertical scale.

Table 6 shows the central resource–accuracy trade-off. Replacing standard convolutions

with depthwise and pointwise steps, and replacing the flattened classifier input with global average pooling, makes Model B dramatically smaller while giving up some test accuracy, macro precision, and macro recall.

Model	Parameters	Size (KB)	Epoch (s)	Accuracy (%)	Precision (%)	Recall (%)
Model A	692,106	2,708.06	11.80	92.89	93.20	92.52
Model B	36,968	150.98	11.61	83.75	83.30	83.42

Table 6: Custom-model comparison generated from the committed test metrics and profiler outputs. Precision and recall are macro averages over the ten EuroSAT classes; size is the saved model file, with 1 KB = 1024 bytes; epoch time was measured on the same Tesla T4.

The storage saving is substantial, but the reduction in training time per epoch is small. The profiler measures computation, while actual training time also depends on data loading, augmentation, framework overhead, memory access, and GPU efficiency. These factors were not measured separately, so they are a likely explanation rather than a measured one. Depthwise convolution reduces computation but does not guarantee a proportional speed improvement on a GPU. Its benefits are clearer under the assignment's edge-device constraints, where fewer parameters reduce storage requirements and fewer MACs reduce computation per inference. Section 6.2 explains why these savings alone do not prove faster execution on the target device.

The confusion matrices show the types of mistakes made by each model. Model A's largest errors (Figure 5) occur when Industrial areas are predicted as Residential areas and Permanent Crop areas are predicted as Herbaceous Vegetation. Model B (Figure 6) makes more mistakes between land-cover categories that look similar, including Herbaceous Vegetation, Permanent Crop, Highway, and River. Both models perform well on classes with more distinct appearances, such as Forest, Residential, and Sea/Lake. These results are based on 30 epochs of training. Section 4.1.1 shows that longer training reduces much of the accuracy gap between Model B and Model A. Therefore, Model B's additional classification errors should not be explained by its smaller model size alone.

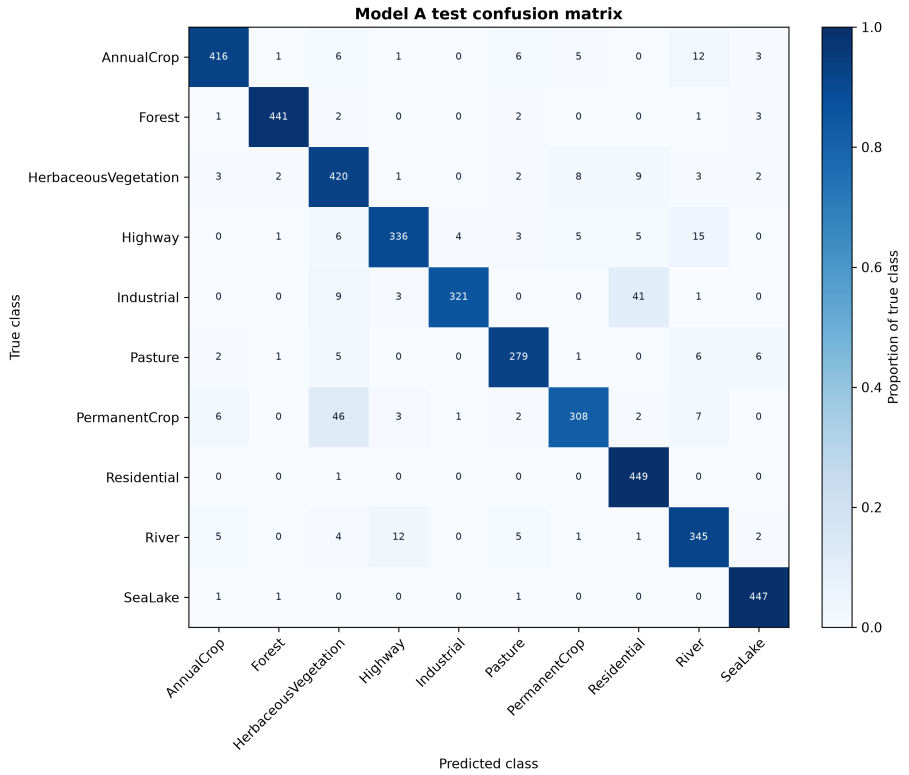


Figure 5: Model A test confusion matrix. Colour is normalised within each true-class row and annotations give raw sample counts.

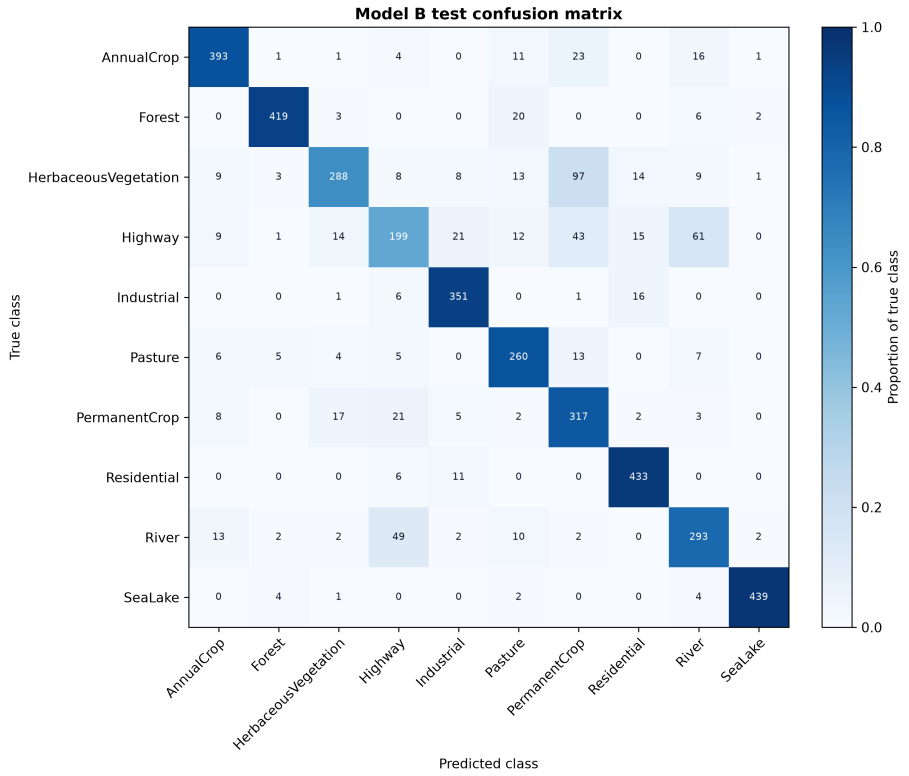


Figure 6: Model B test confusion matrix, using the same convention as Figure 5.

4.1.1 Convergence control

The 30-epoch comparison controls the early training budget, but its loss curve (Figure 4) shows that Model B had not converged. To separate convergence speed from architecture capability, both models were therefore repeated for 100 epochs with otherwise identical configurations (Figure 7). The first 30 loss and accuracy records reproduce the original runs exactly (apart from wall-clock noise), so this is a controlled extension rather than a new hyperparameter search.

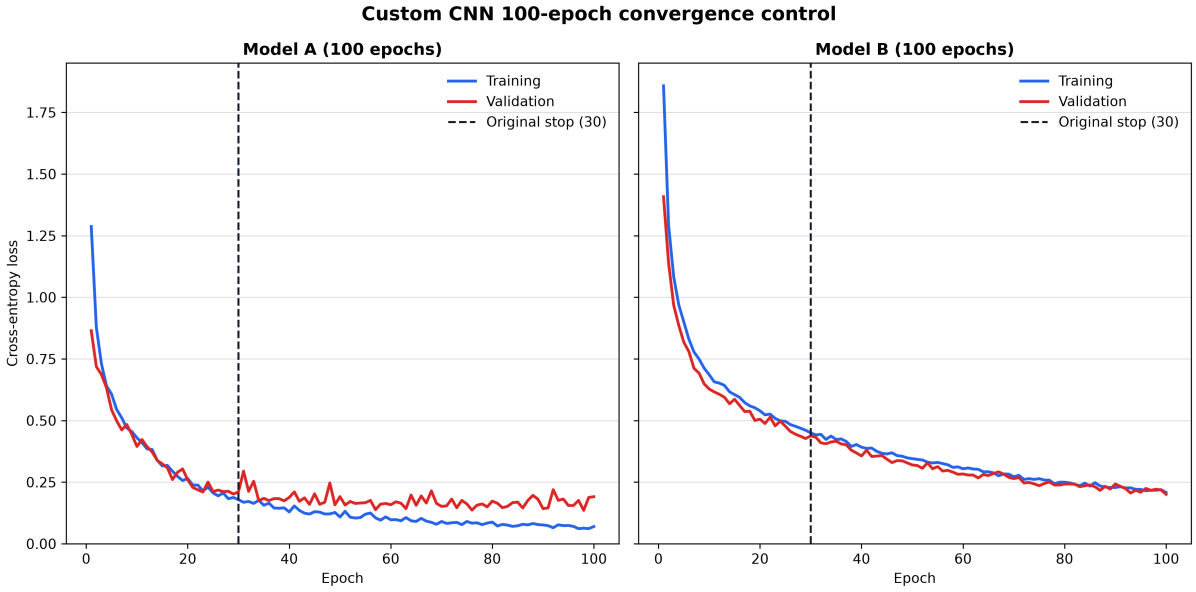


Figure 7: One-hundred-epoch training and validation loss for both custom models. The dashed line marks the original 30-epoch stopping point.

Model	Epochs	Selected	Best val. loss	Best val. acc. (%)	Test acc. (%)	Parameters	Size (KB)	MACs	Epoch (s)
Model A	30	29	0.2018	93.43	92.89	692,106	2,708.06	30,038,016	11.80
Model A	100	98	0.1355	96.00	96.00	692,106	2,708.06	30,038,016	12.54
Model B	30	29	0.4270	84.42	83.75	36,968	150.98	3,725,196	11.61
Model B	100	100	0.2004	93.26	93.14	36,968	150.98	3,725,196	12.29

Table 7: Controlled custom-model convergence comparison. "Selected" is the lowest-validation-loss checkpoint; size uses 1 KB = 1024 bytes; MACs are per image; all timings were measured on the same Tesla T4.

Table 7 shows that longer training benefits Model B much more strongly, substantially narrowing the accuracy gap without changing its parameter, storage, or MAC advantage. The two tables answer different questions: Table 6 measures performance under the same early epoch budget, whereas Table 7 measures the architectures after a longer equal budget. The former must not be presented as Model B's converged capability.

Model A shows mild late-stage overfitting: its training loss continues to fall while validation loss plateaus and fluctuates. Selecting the lowest-validation-loss checkpoint at epoch 98 mitigates that deterioration, and the matching validation and test accuracies support good generalisation of the selected checkpoint. Model B selects the final epoch, so even the 100-epoch result should be described as the achieved result, not a proven upper limit on the architecture.

Overall, Model A is preferable when predictive performance is the priority; Model B is the defensible choice when the parameter ceiling and deployability are hard constraints.

5 Task 5: Lightweight State-of-the-Art Models

5.1 Transfer-learning protocol

Two pretrained architectures were adapted to EuroSAT: MobileNetV2 [7], and EfficientNet-B0 [10]. Both are initialized with ImageNet weights and the final linear layer of each classifier was replaced by a new ten-class prediction layer. Every run uses the same stratified split as the Models A and B, random seed, augmentation, training-split normalization statistics, batch size of 128, 30-epoch budget and Colab Tesla T4 as the machine to run on. The checkpoint was chosen by lowest validation loss, before testing.

After adding the new prediction layer, there were two cases to choose from, either keep the pretrained backbone *frozen*, or *fine-tune* it. Each network was tested with using both methods, to decide wether there is a gain by fine tuning all parameters.

- **Frozen.** In this case, only the new prediction layer was trained, with Adam Optimizer at a learning rate of 10^{-3}. The CNN backbone was used as a fixed feature extractor, with its batch-normalisation statistics held at their original ImageNet values.
- **Fine-tuned.** In this case, every weight was trained, starting from the ImageNet weights, with Adam using a learning rate of 10^{-4}. Here we chose a smaller learning rate than the frozen-backbone rate to limit changes to the pretrained filters while the new classifier learns.

Our main runs keep the input at the original resolution that the custom models see, which is 64×64. Because ImageNet networks are designed for 224×224 input, the experiment was repeated for each network with the 64×64 images upsampled to 224×224, for both the frozen and fine-tuned cases. Each 224×224 pair differs from its original pair in input size only, so as to identify the effect of keeping the input images at the original resolution that the pretrained networks were trained on.

5.2 Results

Model	Backbone	Input	Accuracy (%)	Precision (%)	Recall (%)	Parameters	Trained	Size (MB)
MobileNetV2	Frozen	64×64	93.04	92.80	92.90	2,236,682	12,810	8.77
MobileNetV2	Fine-tuned	64×64	98.02	97.92	97.94	2,236,682	2,236,682	8.77
EfficientNet-B0	Frozen	64×64	87.33	87.02	86.97	4,020,358	12,810	15.63
EfficientNet-B0	Fine-tuned	64×64	98.10	98.04	97.99	4,020,358	4,020,358	15.63
Supplementary: input resolution ablation								
MobileNetV2	Frozen	224×224	96.15	95.99	95.99	2,236,682	12,810	8.77
MobileNetV2	Fine-tuned	224×224	98.84	98.80	98.81	2,236,682	2,236,682	8.77
EfficientNet-B0	Frozen	224×224	92.05	91.94	91.72	4,020,358	12,810	15.63
EfficientNet-B0	Fine-tuned	224×224	99.04	99.04	98.97	4,020,358	4,020,358	15.63

Table 8: Pretrained lightweight models on the EuroSAT test split. Precision and recall are averages over the ten classes. *Trained* counts the parameters updated during training. Sizes are in MB, measured from the saved weights; the custom models in Table 6 are in KB. The rows under Supplementary section upsamples the 64×64 images to 224×224.

Table 8 shows results obtained with both the frozen and fine-tuned configurations for each of the two networks.*Parameters* and *Size* are the same for the frozen and fine-tuned runs of the same network. Freezing a configuration only changes which parameters are trained, and threfore the difference is in the *Trained* column. Accuracy gaps between the two rows of a pair is therefore obtained with training effort only, with no costs in the saved model's size. Fine-tuning does need more memory during training, because gradients are computed for every layer.

We observed that for both networks that accuracy gap is large. Fine-tuning raises all the metrics by well above the frozen feature extractor CNN case. The likely explanation is the domain shift from ImageNet to Satellite Images. ImageNet filters were learnt on close-range photographs of objects, while EuroSAT is overhead satellite images at a much smaller scale. A frozen backbone can only re-weight features learnt for the wrong kind of image, and a fine-tuned configuration can let the filters evolve to field boundaries, road textures and water.

Frozen, EfficientNet-B0 was found to be the weaker network, despite having almost twice MobileNetV2's parameters. In the Fine-tuned case, the two networks are almost at the same level. The gap between the frozen runs shows how well each network's learned ImageNet features happen to transfer to 64×64 satellite images. Once MobileNetV2 is fine-tuned, it is clearly the better edge candidate of the two.

The supplementary rows show that increasing the resolution from 64×64 to 224×224 gives both frozen backbones a gain of few additional points, but the fine-tuned models gain less than one point. A possible explanation for this is the fact that frozen network's filters are designed to fit ImageNet's scale, so upsampling the input is the only way to bring the data to them. Instead, fine-tuning moves the filters to the data, after which resolution becomes less important. For each of the two networks, the fine-tuned 64×64 model outperforms the frozen 224×224 one. It also does about $(224/64)^2 \approx 12$ times fewer operations per image, because a convolution layers cost depends on the $H \times W$ of its output. For an edge device, fine-tuning at native resolution is the cheaper way to get accuracy.

Even the smallest of these models is one to two orders of magnitude larger than Model B (Table 6). Whether the additional accuracy is worth that storage is the question Section 6 answers.

The training curves (Figures 8–11) show the difference between the two approaches for both networks. Each frozen run converges at a higher loss than its respective fine-tuned one. The frozen EfficientNet-B0 curves are also visibly noisier than MobileNetV2's. After the first few epochs both of its losses fluctuate without converging, and at 224×224 the lowest validation loss happens to fall early, so its selected checkpoint comes early in the run. Meanwhile, the fine-tuned curves of both networks are smooth and settle within a few epochs.

The noise in the frozen EfficientNet-B0 curves appears only when the feature extraction is fixed. Whereas its validation loss oscillates from epoch to epoch, its validation accuracy changes much less. This suggests that the predicted classes barely change and that what changes is mainly the confidence given to the images that the frozen feature layers are unable to separate, although the saved histories contain no per-image predictions to confirm it. Cross-entropy heavily penalises a confidently wrong prediction, so small updates to the prediction layer move the loss on these borderline images much more than they move the accuracy. MobileNetV2's frozen features have fewer misclassified images, and its loss is lower and more stable. The two backbones also terminate in different activation functions. EfficientNet-B0 uses the unbounded SiLU activation function [10], and a small fraction of images produce very large pooled features. MobileNetV2's ReLU6 activation limits each feature at six [7]. This bounded behaviour is discussed in Section 2.2. In a diagnostic check which retrained only the prediction layer (committed as `results/frozen_head_diagnostic`), normalizing those features reduced the fluctuation but did not remove it. Using different learning rates of 3×10^{-4} and 10^{-4} did not remove it either, so neither the feature scale nor the learning rate alone explains it. The practical impact is in checkpoint selection. On a noisy validation curve the lowest-loss epoch is partly random, and it need not be the epoch with the best accuracy. Fine-tuning removes this effect, likely because the backbone adapts its features to EuroSAT, which is another reason the comparison in Section 6 uses the fine-tuned runs.

The confusion matrices of the fine-tuned 64×64 models (Figures 12 and 13) identify the errors in the same few class pairs for both networks. Highway and River are confused in both directions, possibly due to the fact thatat 64×64 both are thin line features a few pixels wide. There

is also a substantial number of errors for the agricultural and vegetation classes(AnnualCrop, PermanentCrop, Pasture and HerbaceousVegetation).

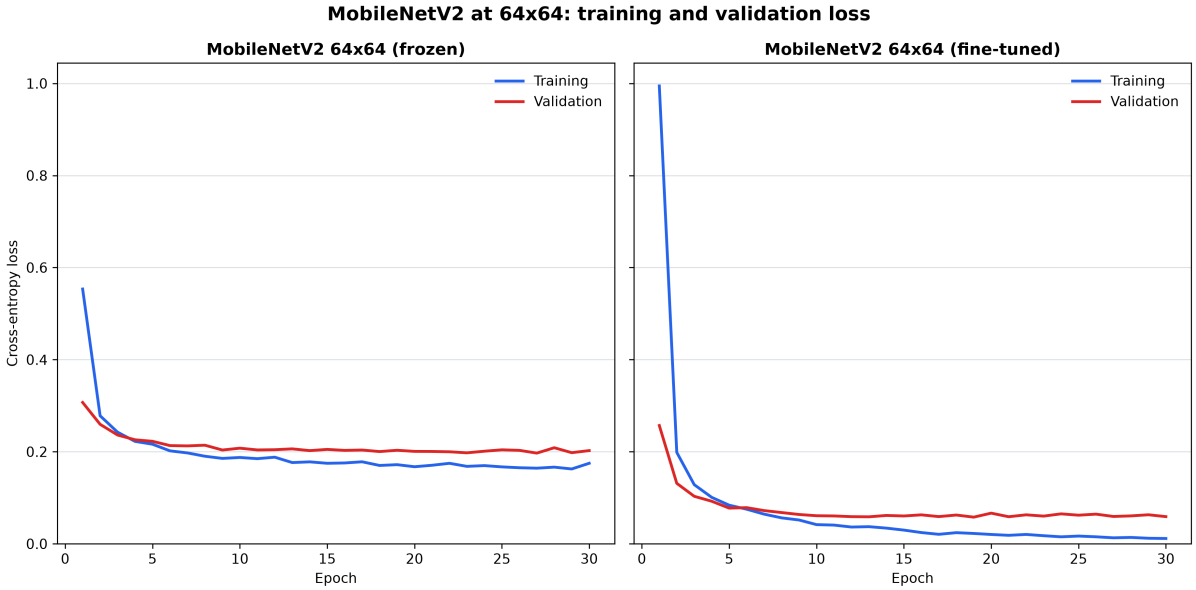


Figure 8: Training and validation loss of MobileNetV2 at 64×64, with the backbone frozen and fine-tuned.

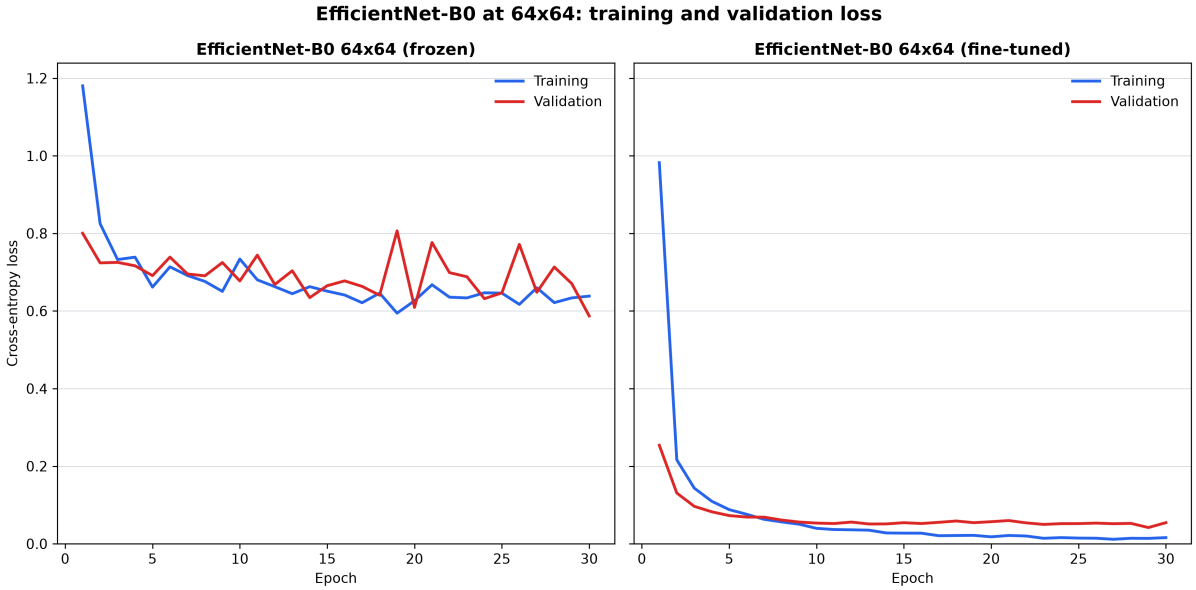


Figure 9: Training and validation loss of EfficientNet-B0 at 64×64, with the backbone frozen and fine-tuned.

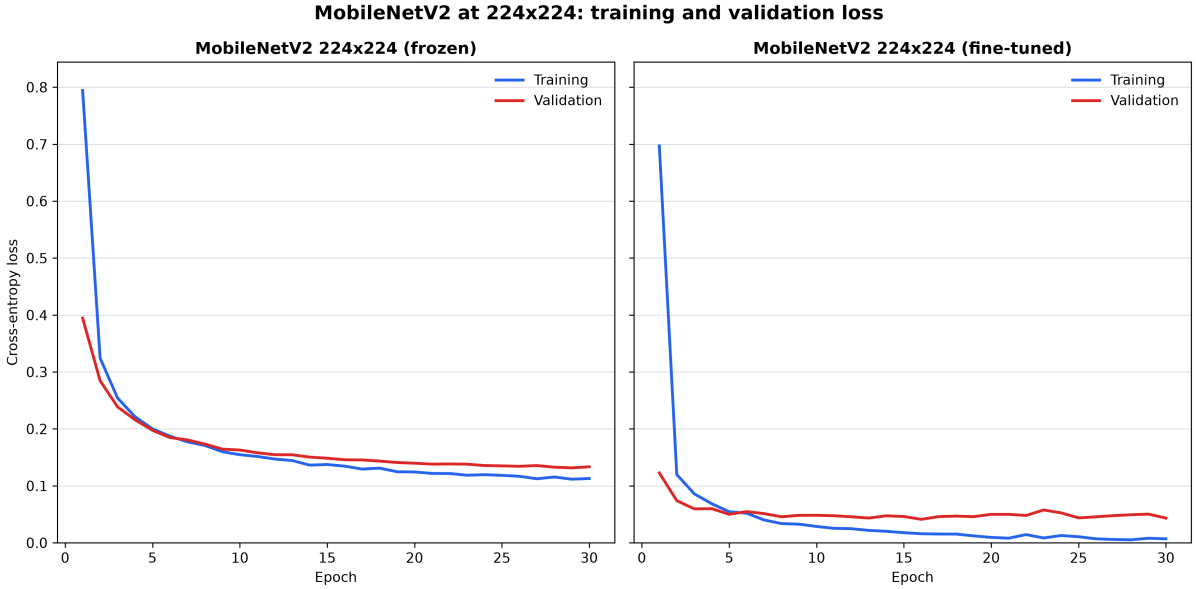


Figure 10: Training and validation loss of MobileNetV2 at 224×224 (the supplementary resolution ablation), with the backbone frozen and fine-tuned.

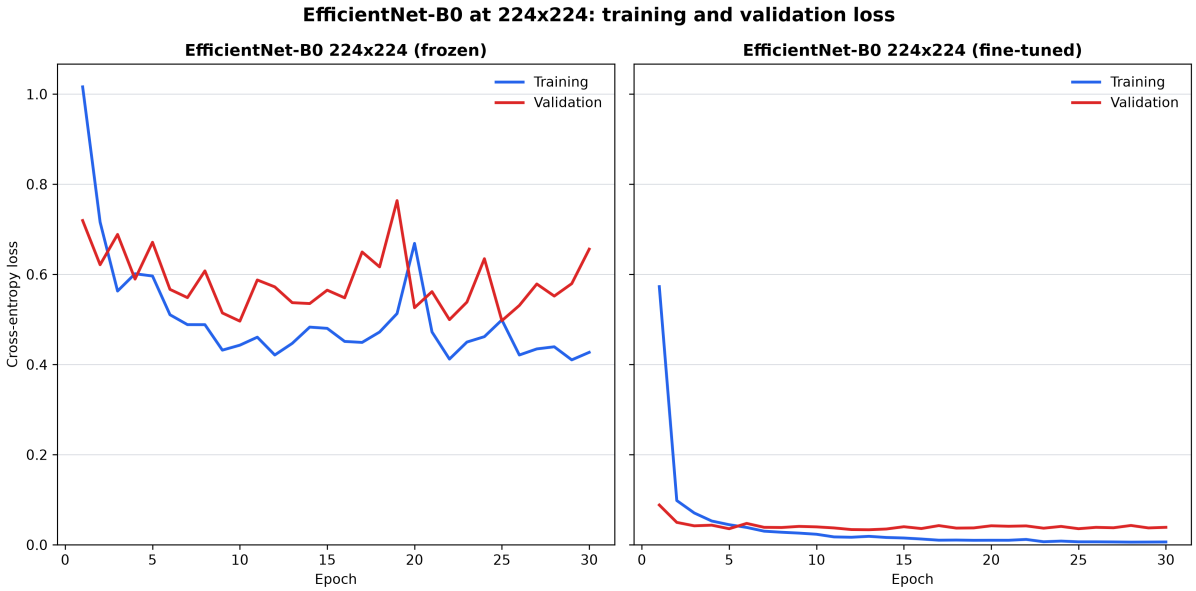


Figure 11: Training and validation loss of EfficientNet-B0 at 224×224 (the supplementary resolution ablation), with the backbone frozen and fine-tuned.

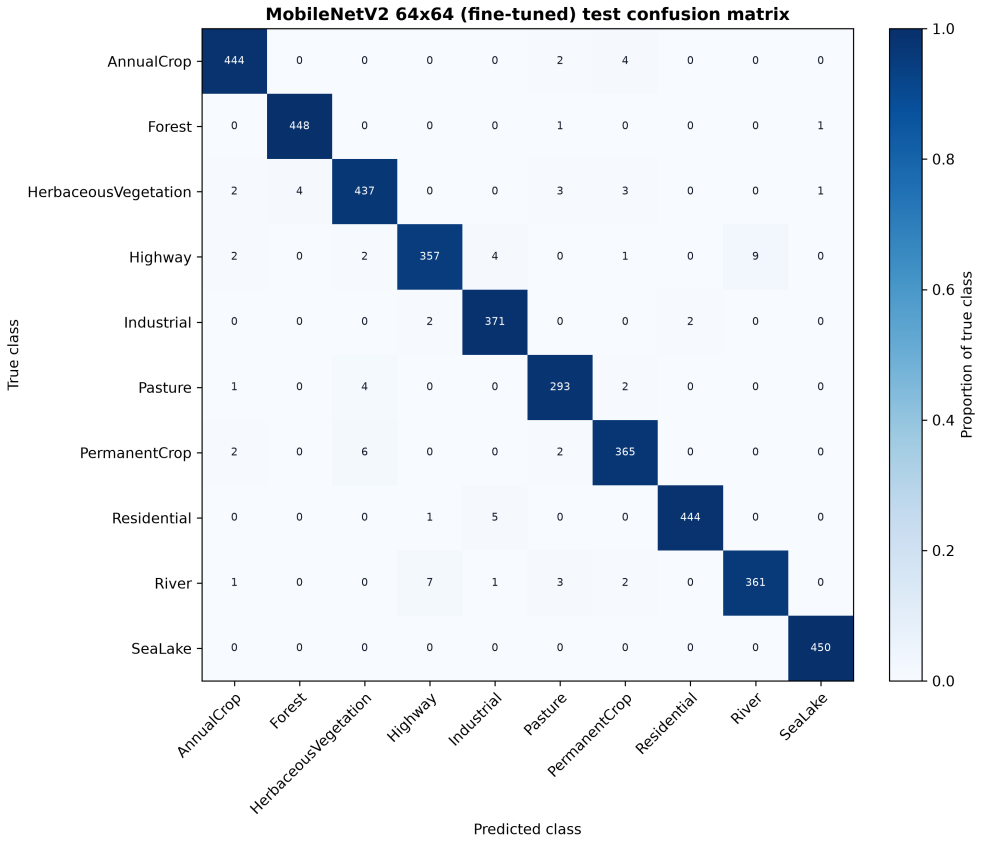


Figure 12: Test confusion matrix of the fine-tuned MobileNetV2 at 64×64.

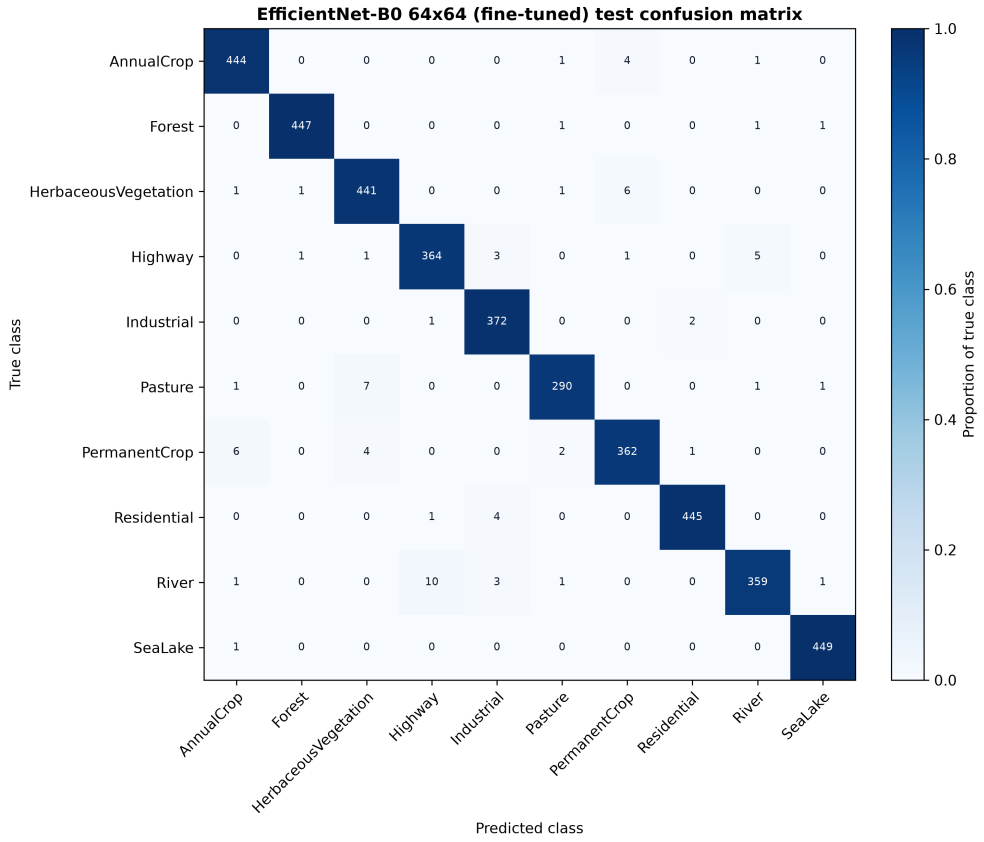


Figure 13: Test confusion matrix of the fine-tuned EfficientNet-B0 at 64×64.

6 Task 6: Final Comparison and Trade-offs

6.1 Model B against lightweight SOTA models

Table 9 compares the longer-trained Model B with the two fully fine-tuned ImageNet models at native 64×64 input. All use the same EuroSAT split, seed, augmentation, batch size and Tesla T4. Each test result belongs to the checkpoint selected by lowest validation loss. The epoch budgets are deliberately visible: this compares achieved models under a deployment constraint, rather than equal training effort. The equal-budget early comparison remains in Table 6; the controlled extension in Table 7 shows why that early Model B result must not be treated as its converged capability.

Model	Epochs	Accuracy (%)	Parameters	Size (KiB)	MACs (M/image)	Epoch (s)
Model B	100	93.14	36,968	150.98	3.725	12.29
MobileNetV2	30	98.02	2,236,682	8,980.07	24.461	14.40
EfficientNet-B0	30	98.10	4,020,358	16,001.55	31.979	16.59

Table 9: Model B trained from scratch against fully fine-tuned SOTA models. Storage is measured saved-state size in KiB (1024 bytes), including serialization overhead; M denotes 10^6 MACs per image. Epoch time is the training pass only on a Tesla T4, not edge inference latency.

6.2 Trade-offs: accuracy, memory and compute

Accuracy and training effort. Model B achieved the accuracy of 93.14% using 36,968 parameters. MobileNetV2 and EfficientNet-B0 reach 98.02% and 98.10%, respectively. This means they performed 4.89 and 4.96 percentage points better than Model B. Even after training Model B for more epochs, MobileNetV2 and EfficientNet-B0 still performed better. However, the difference was much smaller than it appeared in the earlier comparison. Model B's best result came from epoch 100, out of the 100 epochs it was trained for. However, this does not prove that Model B cannot perform better. It only shows the best result achieved in this experiment. More training or different settings might improve its performance. On the other hand, its lower per-epoch time does not make it cheaper to train to this result: multiplying the recorded mean epoch time by the run budget gives 2.85 times the MobileNetV2 epoch-time total and 2.47 times the EfficientNet-B0 total. These estimates exclude previous ImageNet pretraining, validation, final testing, and setup time.

Storage versus working memory. Model B takes up 150.98 KiB on disk. MobileNetV2 and EfficientNet-B0 require 59.5 and 106.0 times more storage, respectively. In comparison, the STM32F407VG microcontroller [11] has 1 MiB of flash memory and 192 KiB of system SRAM. Flash memory stores the model weights, while SRAM is used for temporary data during execution. The SRAM includes 64 KiB of core coupled memory. But, the separate backup SRAM is not included in this comparison. Model B's saved weights are below that flash budget. Neither SOTA saved state is. Firmware and runtime storage must also fit, so this is only a necessary storage condition, not a deployment demonstration.

SRAM is a greater challenge. Model B's first pointwise output as $32 \times 62 \times 62$ is given by the generated layer table in Appendix A. Materialising just that tensor in float32 needs $32 \times 62 \times 62 \times 4 = 492{,}032$ bytes, or about 480.5 KiB, before other live tensors, operator workspace and the stack. It exceeds both the named target's SRAM and a hypothetical 256 KiB SRAM budget even if weights stay in flash. Therefore the unmodified full-tensor implementation has not been shown to fit this microcontroller. A different memory schedule would need separate verification and no embedded implementation or peak-RAM measurement is claimed here.

Arithmetic cost and measured time. In the original image resolution, MobileNetV2 uses 6.57 times and EfficientNet-B0 8.58 times Model B's MACs per image. Depthwise separable convolution uses a two-step construction: for a spatial kernel of width F, K input channels and L output channels, the regular cost per location is F^2KL, whereas depthwise plus pointwise costs $F^2K + KL$. Model B also uses global average pooling to reduce its dense input. However, MobileNetV2 and EfficientNet-B0 already use efficient convolution blocks. Therefore, Model B's additional reduction in computation mainly comes from its smaller overall architecture. The profiler measures MACs from convolutional and linear layers. It does not include activation functions, normalisation, pooling or memory traffic. Therefore, these results allow us to compare the amount of arithmetic computation, but they cannot directly predict actual execution time or energy consumption.

In particular, the differences in training time measured on the Tesla T4 GPU are much smaller than the differences in MAC counts. Training time also depends on data loading, data augmentation and framework overhead. In addition to that, GPUs process different types of operations with different levels of efficiency. These factors may help explain the difference, but they were not measured separately in this study.

What pretraining and resolution establish. Table 8 separates frozen from fully fine-tuned backbones at each resolution. Their improvement at fixed 64×64 supports adapting the pretrained representation to satellite imagery; it does not measure the benefit of ImageNet initialisation in isolation. There is no same-architecture SOTA run trained from random initialisation. Hence, architecture, capacity and pretraining remain confounded in the comparison with Model B. The lower accuracy of frozen models may be due to the difference between ImageNet images and EuroSAT satellite images.

The supplementary 224×224 runs upsample the same native images; they add no new sensor detail. The fine-tuned models at the original resolution achieve higher accuracy than the frozen models using enlarged images. Further, enlarging the images after fine-tuning improves accuracy by less than one percentage point, while also requiring substantially more computation. This is why the final comparison keeps native input and treats upsampling separately: a resolution gain must not be credited to pretraining or presented as free accuracy under an edge compute budget.

Choosing under constraints and limitations. A custom model is attractive for a fixed label set and sensor resolution when storage and arithmetic ceilings dominate. Although the dataset and software licences still apply, its weights are learned directly from the project's dataset, and its architecture can be designed to meet specific requirements without relying on an existing pretrained model. Pretrained networks offer stronger achieved accuracy here and a practical starting point when development time or labelled data are limited, especially for domains closer to their source data. Those general advantages were not isolated by a data-size or development-time experiment in this study.

Accuracy per KiB describes storage efficiency, but cannot replace an application's minimum acceptable accuracy or SRAM and latency constraints. Among the pretrained options, EfficientNet-B0 gains only 0.07 percentage points over MobileNetV2, which corresponds to 3 extra correct test predictions, with substantially larger storage and MAC requirements. Without repeated seeds this small gap does not establish reliable superiority. More generally, the results are based on a single random seed and one dataset split. Therefore, they do not establish whether the models will perform well on images from different geographical locations or other sensors. They also do not measure the uncertainty in the model rankings.

6.3 Conclusion

Model B is the best option when the available parameters and storage are very limited. However, this comes with lower accuracy. Training it for longer reduces the accuracy difference without increasing the resources needed during inference. When more storage is available and accuracy is a higher priority, fine-tuned MobileNetV2 at the original image resolution is the more resource-efficient pretrained option in these experiments. EfficientNet-B0's small observed accuracy gain does not justify its larger footprint without an application-specific need. None of these choices is a verified microcontroller deployment: activation memory, target latency and energy remain unmeasured, even for the sub-100k model.

A Generated Layer Tables

`scripts/build_tables.py` generates these tables from a forward pass of one 64×64 image through each model, and their parameter counts agree with the hand derivations in Section 2. The tables list only operations that change the shape or carry parameters, so ReLU and dropout do not appear in them. Section 2.1 gives where those two operations sit in each model.

Table 10: Model A layer summary, generated from `configs/model_a.yaml`.

Layer	Output shape ($C \times H \times W$)	Parameters
Conv2D 1	$32 \times 62 \times 62$	896
MaxPooling2D 1	$32 \times 31 \times 31$	0
Conv2D 2	$64 \times 29 \times 29$	18,496
MaxPooling2D 2	$64 \times 14 \times 14$	0
Conv2D 3	$128 \times 12 \times 12$	73,856
MaxPooling2D 3	$128 \times 6 \times 6$	0
Flatten	4608	0
Dense 1	128	589,952
Dense 2	64	8,256
Dense 3	10	650
Total		692,106

Table 11: Model B layer summary, generated from `configs/model_b.yaml`. Each depthwise separable block appears as its two steps.

Layer	Output shape ($C \times H \times W$)	Parameters
DepthwiseConv2D 1	$3 \times 62 \times 62$	30
PointwiseConv2D 1	$32 \times 62 \times 62$	128
MaxPooling2D 1	$32 \times 31 \times 31$	0
DepthwiseConv2D 2	$32 \times 29 \times 29$	320
PointwiseConv2D 2	$64 \times 29 \times 29$	2,112
MaxPooling2D 2	$64 \times 14 \times 14$	0
DepthwiseConv2D 3	$64 \times 12 \times 12$	640
PointwiseConv2D 3	$128 \times 12 \times 12$	8,320
MaxPooling2D 3	$128 \times 6 \times 6$	0
GlobalAveragePooling2D	$128 \times 1 \times 1$	0
Flatten	128	0
Dense 1	128	16,512
Dense 2	64	8,256
Dense 3	10	650
Total		36,968

B Training Hyperparameters

Table 12 lists the training settings of every run whose results appear in this report, as recorded in each run's resolved `config.yaml` in the repository.

Three learning rates appear, none tuned for this problem. Adam uses its authors' default of 10^{-3} wherever a network trains from scratch or only a new prediction layer is trained, and SGD uses the common starting value 10^{-2} (Section 3.1). Full fine-tuning uses 10^{-4}, below the frozen-head rate, to limit changes to the pretrained filters (Section 5.1).

Table 12: Training hyperparameters of every reported run. All runs use seed 42, weight decay 0, dropout 0.2, training-set flips and quarter-turns, and a constant learning rate. Momentum applies to SGD only.

Run	Model	Backbone	Optimizer	LR	Momentum	Input	Batch	Epochs
Custom models, Adam (Tasks 3 and 4)								
`model_a`	Model A	–	Adam	10^{-3}	–	64×64	128	30
`model_a_100`	Model A	–	Adam	10^{-3}	–	64×64	128	100
`model_b`	Model B	–	Adam	10^{-3}	–	64×64	128	30
`model_b_100`	Model B	–	Adam	10^{-3}	–	64×64	128	100
Custom Model B, SGD (Task 3 optimizer and momentum study)								
`model_b_sgd`	Model B	–	SGD	10^{-2}	0	64×64	128	30
`model_b_sgdm`	Model B	–	SGD	10^{-2}	0.9	64×64	128	30
`momentum_m000`	Model B	–	SGD	10^{-2}	0	64×64	128	30
`momentum_m050`	Model B	–	SGD	10^{-2}	0.5	64×64	128	30
`momentum_m090`	Model B	–	SGD	10^{-2}	0.9	64×64	128	30
`momentum_m099`	Model B	–	SGD	10^{-2}	0.99	64×64	128	30
Pretrained models (Task 5)								
`mobilenetv2_64`	MobileNetV2	frozen	Adam	10^{-3}	–	64×64	128	30
`mobilenetv2_64_ft`	MobileNetV2	fine-tuned	Adam	10^{-4}	–	64×64	128	30
`efficientnetb0_64`	EfficientNet-B0	frozen	Adam	10^{-3}	–	64×64	128	30
`efficientnetb0_64_ft`	EfficientNet-B0	fine-tuned	Adam	10^{-4}	–	64×64	128	30
`mobilenetv2_224`	MobileNetV2	frozen	Adam	10^{-3}	–	224×224	128	30
`mobilenetv2_224_ft_b128`	MobileNetV2	fine-tuned	Adam	10^{-4}	–	224×224	128	30
`efficientnetb0_224`	EfficientNet-B0	frozen	Adam	10^{-3}	–	224×224	128	30
`efficientnetb0_224_ft_b128`	EfficientNet-B0	fine-tuned	Adam	10^{-4}	–	224×224	128	30

References

[1] P. Helber, B. Bischke, A. Dengel, and D. Borth, "EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 7, pp. 2217–2226, Jul. 2019, doi: 10.1109/JSTARS.2019.2918242.

[2] TorchVision maintainers and contributors, "TorchVision: PyTorch's computer vision library," GitHub repository, 2016. [Online]. Available: https://github.com/pytorch/vision. Accessed: Oct. 19, 2026.

[3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Miami, FL, USA, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848.

[4] Arm Ltd., "CMSIS-NN software library," GitHub repository. Revision `71cbe3d5a686c875c88a4d112633b13fae94b5a2`. [Online]. Available: https://github.com/ARM-software/CMSIS-NN/tree/71cbe3d5a686c875c88a4d112633b13fae94b5a2. Accessed: Oct. 19, 2026.

[5] Arm Ltd., "White paper: DSP capabilities of Cortex-M4 and Cortex-M7," Arm Community Blogs, 2016. [Online]. Available: https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/white-paper-dsp-capabilities-of-cortex-m4-and-cortex-m7

[6] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," 2016, arXiv:1606.08415.

[7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520, doi: 10.1109/CVPR.2018.00474.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, San Diego, CA, USA, 2015, arXiv:1412.6980.

[9] R. M. Schmidt, F. Schneider, and P. Hennig, "Descending through a crowded valley—Benchmarking deep learning optimizers," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 139, 2021, pp. 9367–9376.

[10] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 97, Long Beach, CA, USA, 2019, pp. 6105–6114.

[11] STMicroelectronics, "STM32F407VG microcontroller," product page. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32f407vg.html